Lab: Recursion, Inductive Data Types, and Abstract Syntax Trees

Learning Goals

The primary learning goals of this assignment are to build intuition for the following:

Functional Programming Skills Representing data structures using algebraic data types.
Programming Languages Ideas Representing programs as abstract syntax.

Instructions

A version of project files for this lab resides in the public pppl-lab1 repository. Please follow separate instructions to get a private clone of this repository for your work.

You will be replacing `???` in the `Lab1.scala` file with solutions to the coding exercises described below. Make sure that you remove the `???` and replace it with the answer.

You may add additional tests to the `Lab1Spec.scala` file. In the `Lab1Spec.scala`, there is empty test class `Lab1StudentSpec` that you can use to separate your tests from the given tests in the `Lab1Spec` class. You are also likely to edit `Lab1.worksheet.sc` for any scratch work.

Single-file notebooks are convenient when experimenting with small bits of code, but they can become unwieldy when one needs a multiple-file project instead. In this case, we use standard build tools (e.g., `sbt` for Scala), IDEs (e.g., Visual Studio Code with Metals), and source control systems (e.g., `git` with GitHub). While it is almost overkill to use these standard software engineering tools for this lab, we get practice using these tools in the small.

If you like, you may use this notebook for experimentation. However, **please make sure your code is in `Lab1.scala`; this notebook will not graded.**

Recursion

Repeat String

Exercise 0.1. Write a recursive function `repeat`

where `repeat(s, n)` returns a string with `n` copies of `s` concatenated together. For example, `repeat("a",3)` returns `"aaa"`. Implement by this function by direct recursion. Do not use any Scala library methods.

Square Root

In this exercise, we will implement the square root function. To do so, we will use Newton's method (also known as Newton-Raphson).

Recall from Calculus that a root of a differentiable function can be iteratively approximated by following tangent lines. More precisely, let f be a differentiable function, and let x_0 be an initial guess for a root of f. Then, Newton's method specifies a sequence of approximations $x_0, x_1, ...$ with the following recursive equation:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} .$$

The square root of a real number c for $c > 0$, written $\sqrt{c}$, is a positive x such that $x^2 = c$. Thus, to compute the square root of a number c, we want to find the positive root of the function:

$$f(x) = x^2 - c .$$

Thus, the following recursive equation defines a sequence of approximations for $\sqrt{c}$:

$$x_{n+1} = x_n - \frac{x_n^2 - c}{2x_n} .$$

Exercise 0.2. First, implement a function `sqrtStep`

```
def sqrtStep(c: Double, xn: Double): Double = ???
```

that takes one step of approximation in computing $\sqrt{c}$ (i.e., computes x_{n+1} from x_n).

Exercise 0.3. Next, implement a function `sqrtN`

```
def sqrtN(c: Double, x0: Double, n: Int): Double = ???
```

that computes the nth approximation x_n from an initial guess x_0. You will want to call `sqrtStep` implemented in the previous part.

You need to implement this function using recursion and no mutable variables (i.e., **`var`**s)—you will want to use a recursive helper function. It is also quite informative to compare your recursive solution with one using a **`while`** loop.

Exercise 0.4. Now, implement a function `sqrtErr`

```
def sqrtErr(c: Double, x0: Double, epsilon: Double): Double = ???
```

that is very similar to `sqrtN` but instead computes approximations x_n until the approximation error is within ε (`epsilon`), that is, $|x_n^2 - c| < \varepsilon$. You can use your absolute value function `abs` implemented in a previous part. A wrapper function `sqrt` is given in the template that simply calls `sqrtErr` with a choice of `x0` and `epsilon`.

You need to implement this function using recursion, though it is useful to compare your recursive solution to one with a **`while`** loop.

Data Structures Review: Binary Search Trees

In this question, we review implementing operations on binary search trees from Data Structures. Balanced binary search trees are common in standard libraries to implement collections, such as sets or maps. For simplicity, we do not worry about balancing in this question.

Trees are important structures in developing interpreters, so this question is also critical practice in implementing tree manipulations.

A binary search tree is a binary tree that satisfies an ordering invariant. Let n be any node in a binary search tree whose left child is l, data value is d, and right child is r. The ordering invariant is that all of the data values in the subtree rooted at l must be $< d$, and all of the data values in the subtree rooted at r must be $\geq d$. We will represent a binary trees containing integer data using the following Scala **`case class`**es:

```
sealed trait Tree
case object Empty extends Tree
case class Node(l: Tree, d: Int, r: Tree) extends Tree
```

A `Tree` is either `Empty` or a `Node` with left child `l`, data value `d`, and right child `r`.

For this question, we will implement the following four functions.

Exercise 0.5. The function `repOk`

```
def repOk(t: Tree): Boolean = {
  def check(t: Tree, min: Int, max: Int): Boolean = t match {
    case Empty => true
    case Node(l, d, r) => ???
  }
  check(t, Int.MinValue, Int.MaxValue)
}
```

checks that an instance of `Tree` is valid binary search tree. In other words, it checks using a traversal of the tree the ordering invariant described above. This function is useful for testing your implementation.

Exercise 0.6. The function `insert`

```
def insert(t: Tree, n: Int): Tree = ???
```

inserts an integer into the binary search tree. Observe that the return type of `insert` is a `Tree`. This choice suggests a functional style where we construct and return a new output tree that is the input tree `t` with the additional integer `n` as opposed to destructively updating the input tree.

Exercise 0.7. The function `deleteMin`

```
def deleteMin(t: Tree): (Tree, Int) = {
  require(t != Empty)
  (t: @unchecked) match {
    case Node(Empty, d, r) => (r, d)
    case Node(l, d, r) =>
      val (l1, m) = deleteMin(l)
      ???
  }
}
```

deletes the smallest data element in the search tree (i.e., the leftmost node). It returns both the updated tree and the data value of the deleted node. This function is intended as a helper function for the `delete` function.

Exercise 0.8. The function `delete`

```
def delete(t: Tree, n: Int): Tree = ???
```

removes the first node with data value equal to `n`. This function is trickier than `insert` because what should be done depends on whether the node to be deleted has children or not. We advise that you take advantage of pattern matching to organize the cases.

Interpreter: JavaScripty Calculator

In this question, we consider the arithmetic sub-language of JavaScripty (i.e., a basic calculator). We represent the abstract syntax for this sub-language in Scala using the following inductive data type:

Listing 1 Representing in Scala the abstract syntax of the arithmetic sub-language of JavaScripty (see `ast.scala`).

```
sealed trait Expr                                          // e ::=
case class N(n: Double) extends Expr                       //   n
case class Unary(uop: Uop, e1: Expr) extends Expr          // | uop e1
case class Binary(bop: Bop, e1: Expr, e2: Expr) extends Expr // | e1 bop e2

sealed trait Uop              // uop ::=
case object Neg extends Uop   //   -

sealed trait Bop              // bop ::=
case object Plus extends Bop  //   +
case object Minus extends Bop // | -
case object Times extends Bop // | *
case object Div extends Bop   // | /
```

In comments, we give a grammar that connects the abstract syntax with the concrete syntax of the language. We consider grammars in more detail subsequentlyin **?@sec-grammars**. For now, it is fine to ignore the concrete syntax or use your intuition for the connection. ow, given the inductive data type `Expr` defining the abstract syntax:

Exercise 0.9. Implement the `eval` function

```
def eval(e: Expr): Double = e match {
  case N(n) => ???
  case _  => ???
}
```

that evaluates the Scala representation of a JavaScripty expression `e` to the Scala double-precision floating point number corresponding to the Scala representation of the JavaScripty *value* of `e`. At this point, you have implemented your first language interpreter!

To go in more detail, consider a JavaScripty expression e, and imagine e to be concrete syntax. This text is parsed into a JavaScripty AST `e`, that is, a Scala value of type `Expr`. Then, the result of `eval` is a Scala number of type `Double` and should match the interpretation of e as a JavaScript expression. These distinctions can be subtle but learning to distinguish between them will go a long way in making sense of programming languages.

To see what a JavaScripty expression e should evaluate to, you may want to run e through a JavaScript interpreter to see what the value should be. For example,

```
3 + 4
```

```
1 / 7
```

```
6 * 4 - 2 + 10
```

Experiment in a Worksheet

Scala worksheets provide an interactive interface in the context of a multi-file project. A worksheet is a good place to start for experimenting with an implementation, whether on existing code or code that you are in the process of writing. A scratch worksheet `Lab1.worksheet.sc` is provided for you in the code repository.

To test and experiment with your `eval` function, you can write JavaScripty expressions directly in abstract syntax like above. You can also make use of a parser that is provided for you: it reads in a JavaScripty program-as-a-`String` and converts into an abstract syntax tree of type `Expr`.

For your convenience, we have also provided in the template `Lab1.scala` file, an overloaded `eval: String => Double` function that calls the provided parser and then delegates to your `eval: Expr => Double` function.

Test-Driven Development and Regression Testing

Once you have experimented enough in your worksheet to have some tests, it is useful to save those tests to run over-and-over again as you work on your implementation. The idea behind test-driven development is that we first write a test for what we expect our implementation to do. Initially, we expect our implementation to fail the test, and then we work on our implementation until the test succeeds. IDEs have features to support this workflow. While a test suite can never be exhaustive, we have provided a number of initial tests for you in `Lab1Spec.scala` to partially drive your test-driven development of the functions in this assignment.

Additional Notes

While you may not need them in this assignment, the `ast.scala` file also includes some basic helper functions for working with the AST, such as

```
def isValue(e: Expr): Boolean = e match {
  case N(_) => true
  case _ => false
}

val e_minus4_2 = N(-4.2)
isValue(e_minus4_2)

val e_neg_4_2 = Unary(Neg, N(4.2))
isValue(e_neg_4_2)
```

the defines which expressions are values. In this case, literal number expressions `N(` n `)` are values where n is the meta-variable for JavaScripty numbers. We represent JavaScripty numbers in Scala with Scala values of type `Double`.

We also define functions to pretty-print, that is, convert abstract syntax to concrete syntax:

```
def prettyNumber(n: Double): String =
  if (n.isWhole) "%.0f" format n else n.toString

def pretty(v: Expr): String = {
  require(isValue(v))
  (v: @unchecked) match {
    case N(n) => prettyNumber(n)
  }
}

pretty(N(4.2))
pretty(N(10))
```

We only define `pretty` for values, and we do not override the `toString` method so that the abstract syntax can be printed as-is.

```
e_minus4_2.toString
e_neg_4_2.toString
```

The `@unchecked` annotation tells the Scala compiler that we know the pattern match is non-exhaustive syntactically, so we do not want to be warned about it. However, we see that our definition of `isValue` rules out the potential for a match error at run time (right?).

Submission

If you are a University of Colorado Boulder student, we use Gradescope for assignment submission. In summary,

- ☐ Create a private GitHub repository by clicking on a GitHub Classroom link from the corresponding Canvas assignment entry.
- ☐ Clone your private GitHub repository to your development environment (using the `<> Code` button on GitHub to get the repository URL).
- ☐ Work on this lab from your cloned repository. Use Git to save versions on GitHub (e.g., `git add`, `git commit`, `git push` on the command line or via VSCode).
- ☐ Submit to the corresponding Gradescope assignment entry for grading by choosing GitHub as the submission method.

You need to have a GitHub identity and must have your full name in your GitHub profile in case we need to associate you with your submissions.